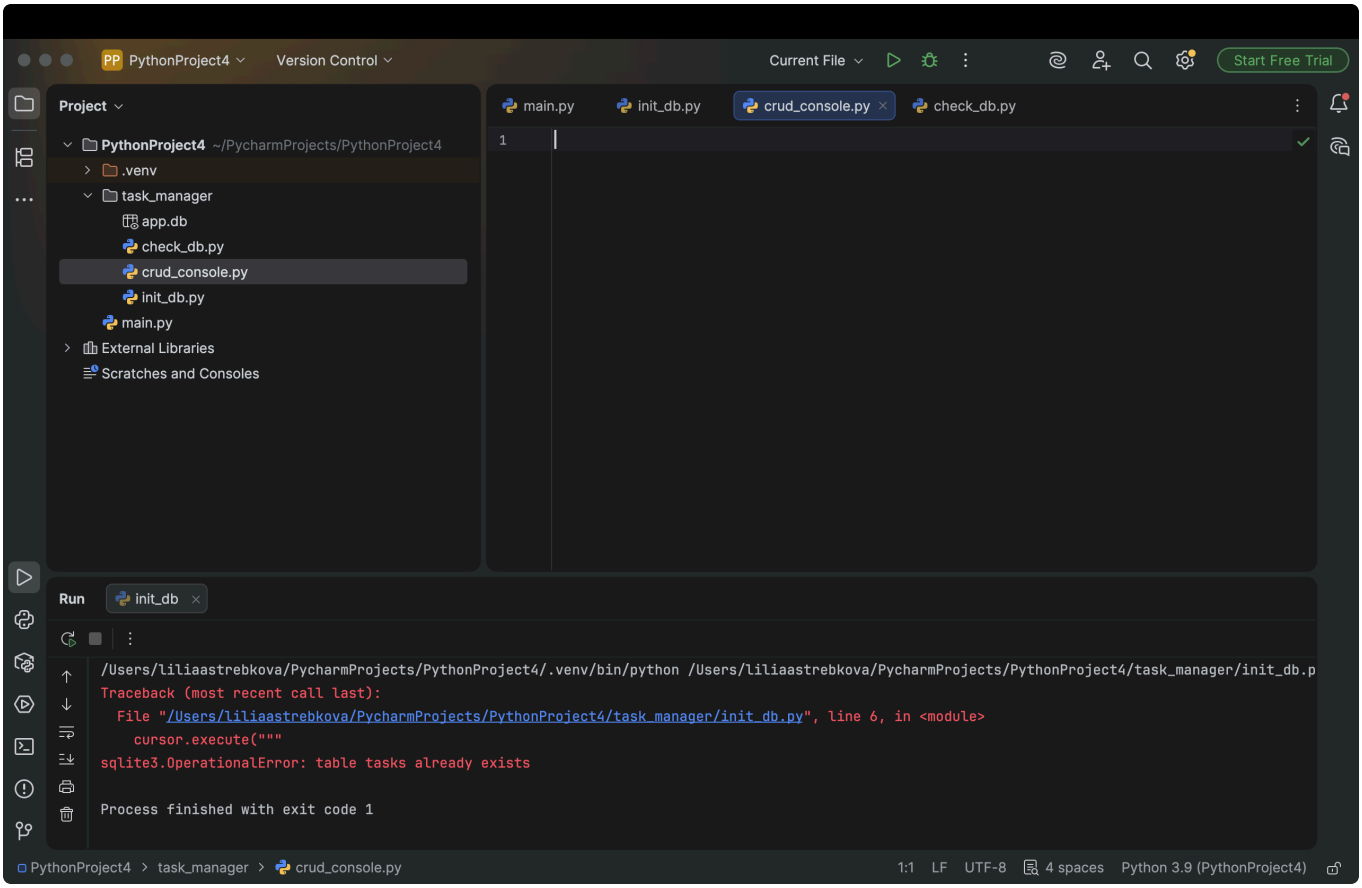
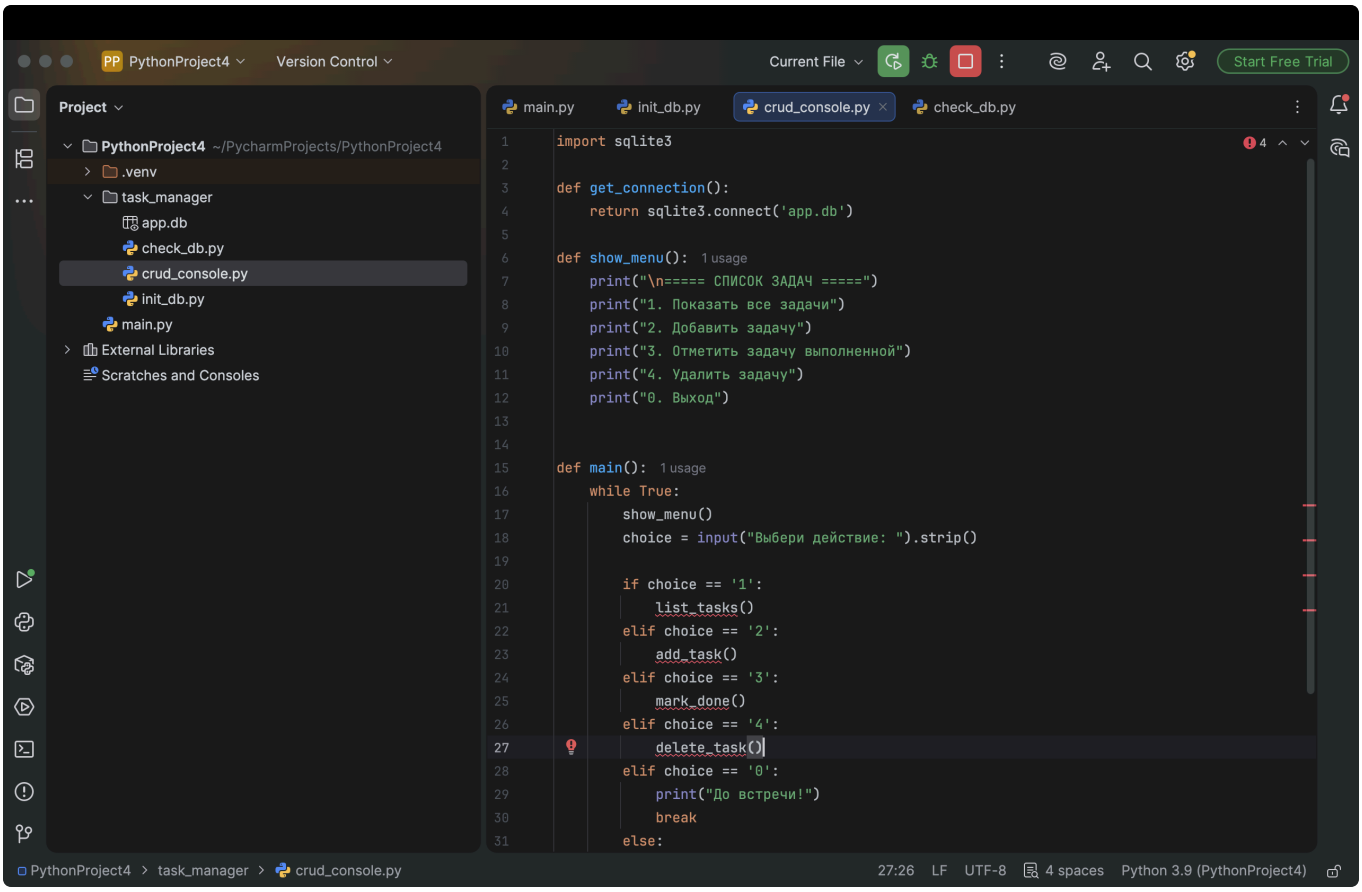
Базовые CRUD-операции через консоль

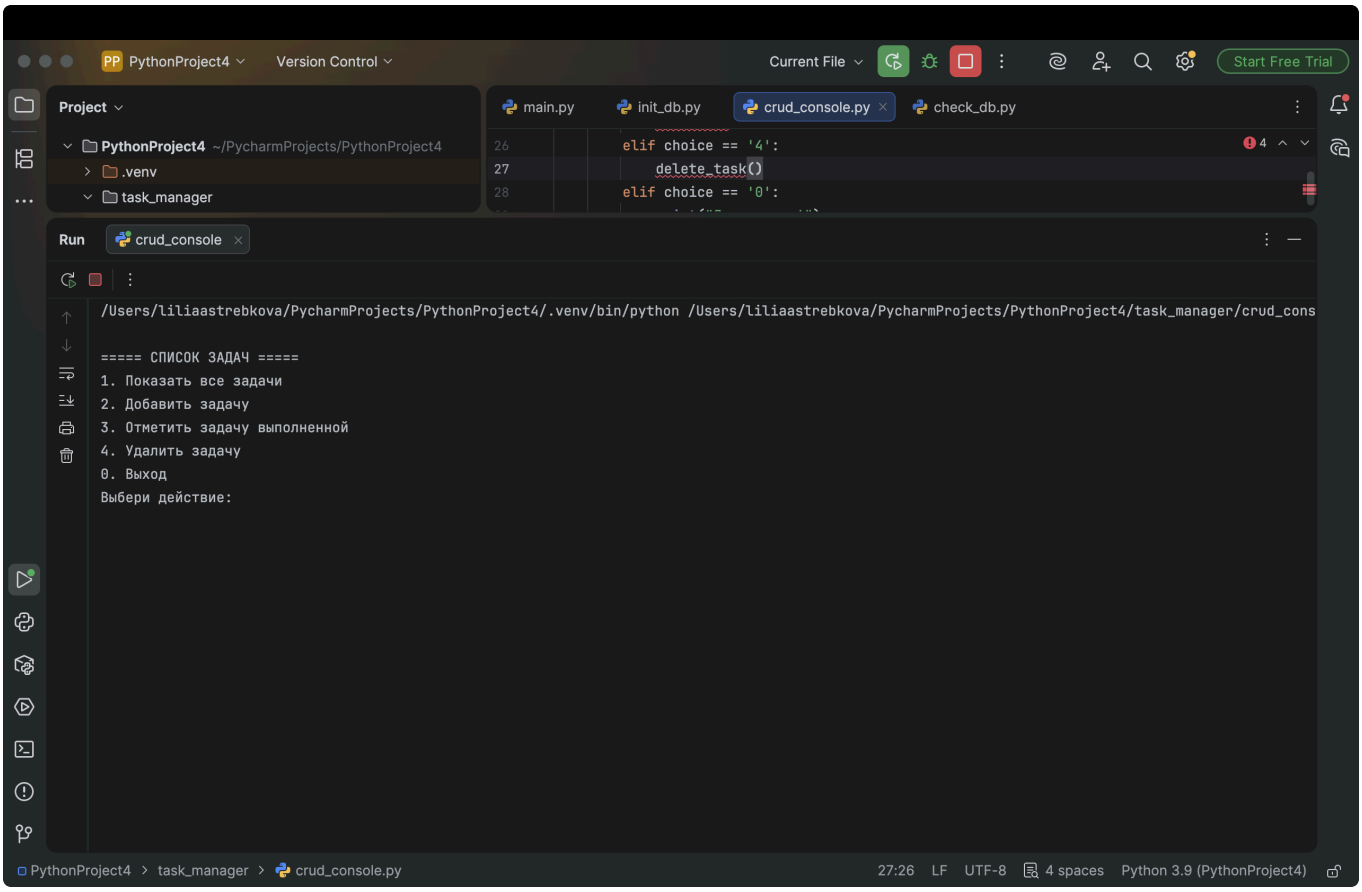
Ястребкова Лилия ЗИСП11-44

Шаг 1. Подготовка

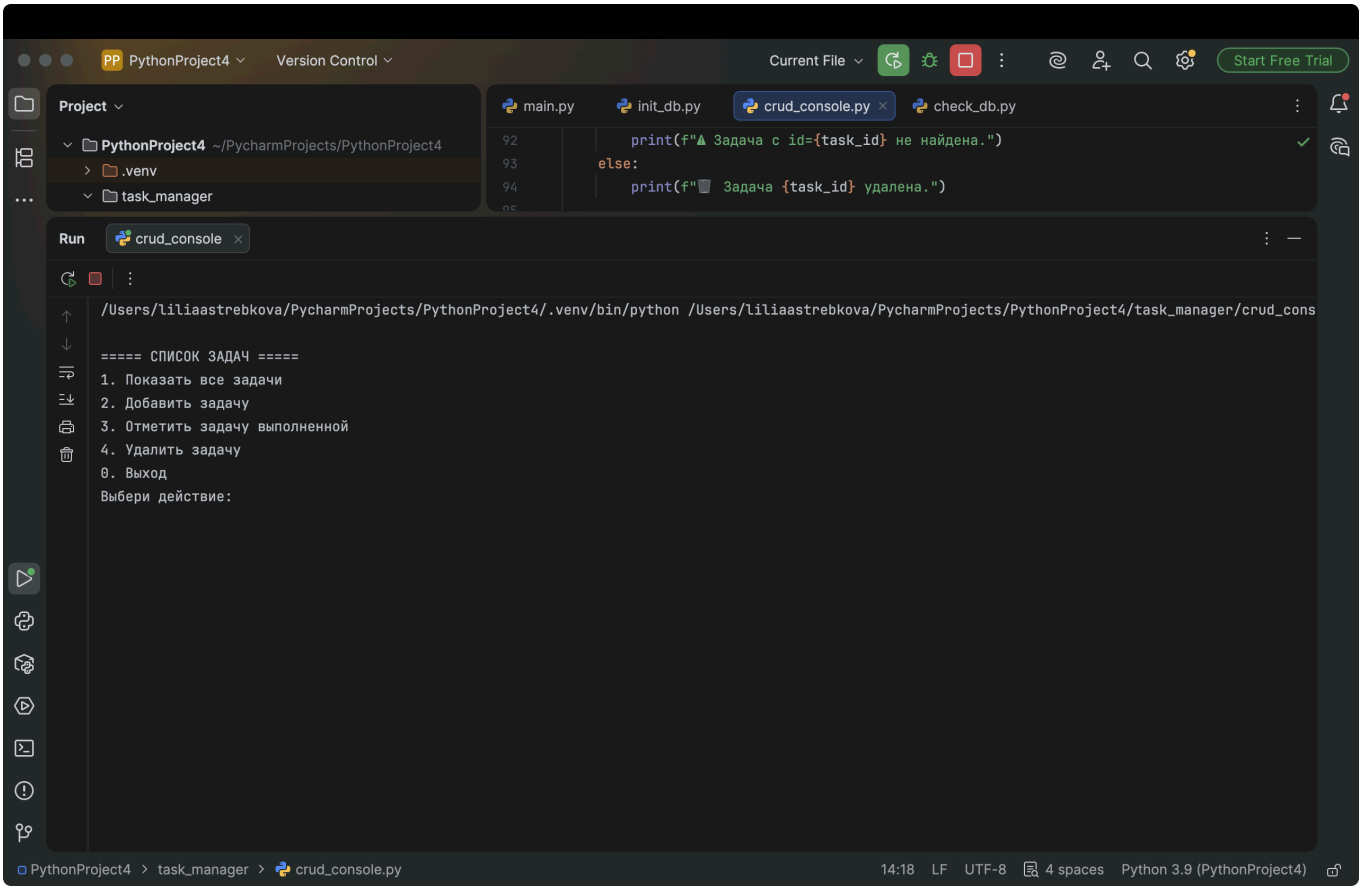


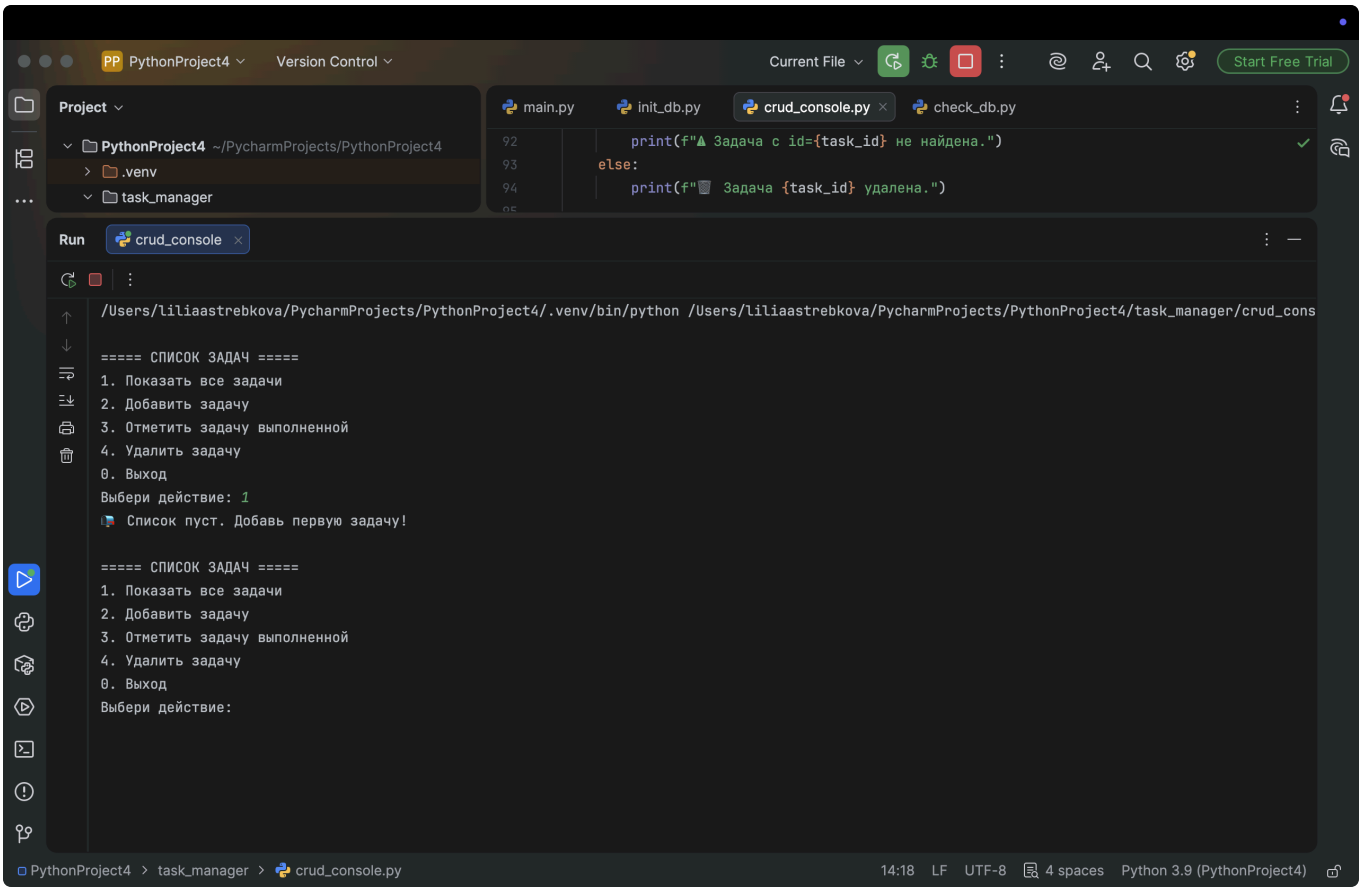
Шаг 2. Каркас приложения с меню

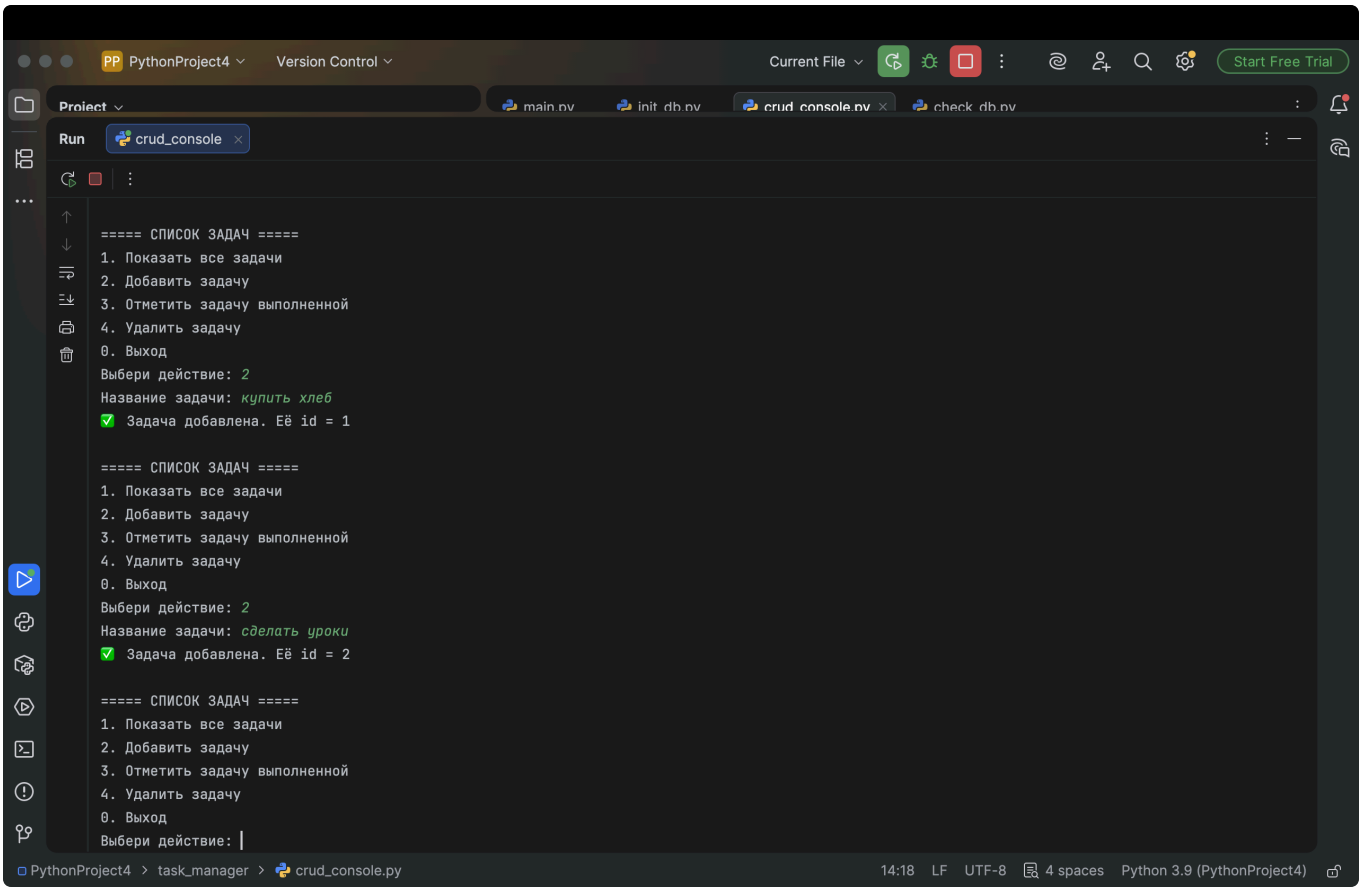

PythonProject4
Version Control
Current File
Start Free Trial
Project
PythonProject4 ~/PycharmProjects/PythonProject4
.venv
task_manager
app.db
check_db.py
crud_console.py
init_db.py
main.py
External Libraries
Scratches and Consoles
main.py
init_db.py
crud_console.py
check_db.py
import sqlite3
def get_connection():
return sqlite3.connect('app.db')
def show_menu(): 1 usage
print("\n===== СПИСОК ЗАДАЧ =====")
print("1. Показать все задачи")
print("2. Добавить задачу")
print("3. Отметить задачу выполненной")
print("4. Удалить задачу")
print("0. Выход")
def main(): 1 usage
while True:
show_menu()
choice = input("Выбери действие: ").strip()
if choice == '1':
list_tasks()
elif choice == '2':
add_task()
elif choice == '3':
mark_done()
elif choice == '4':
delete_task()
elif choice == '0':
print("До встречи!")
break
else:
PythonProject4 > task_manager > crud_console.py
27:26 LF UTF-8 4 spaces Python 3.9 (PythonProject4)

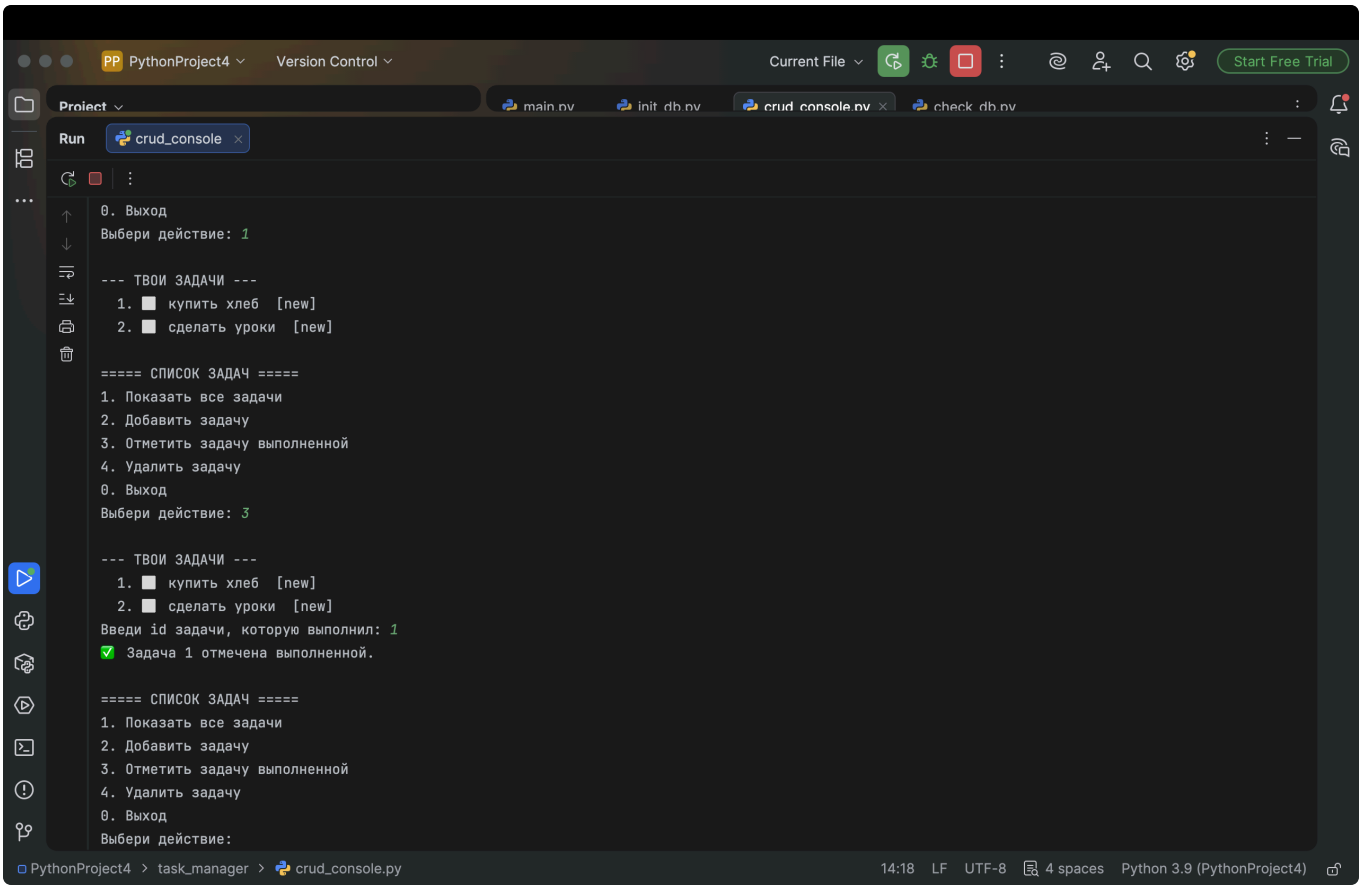


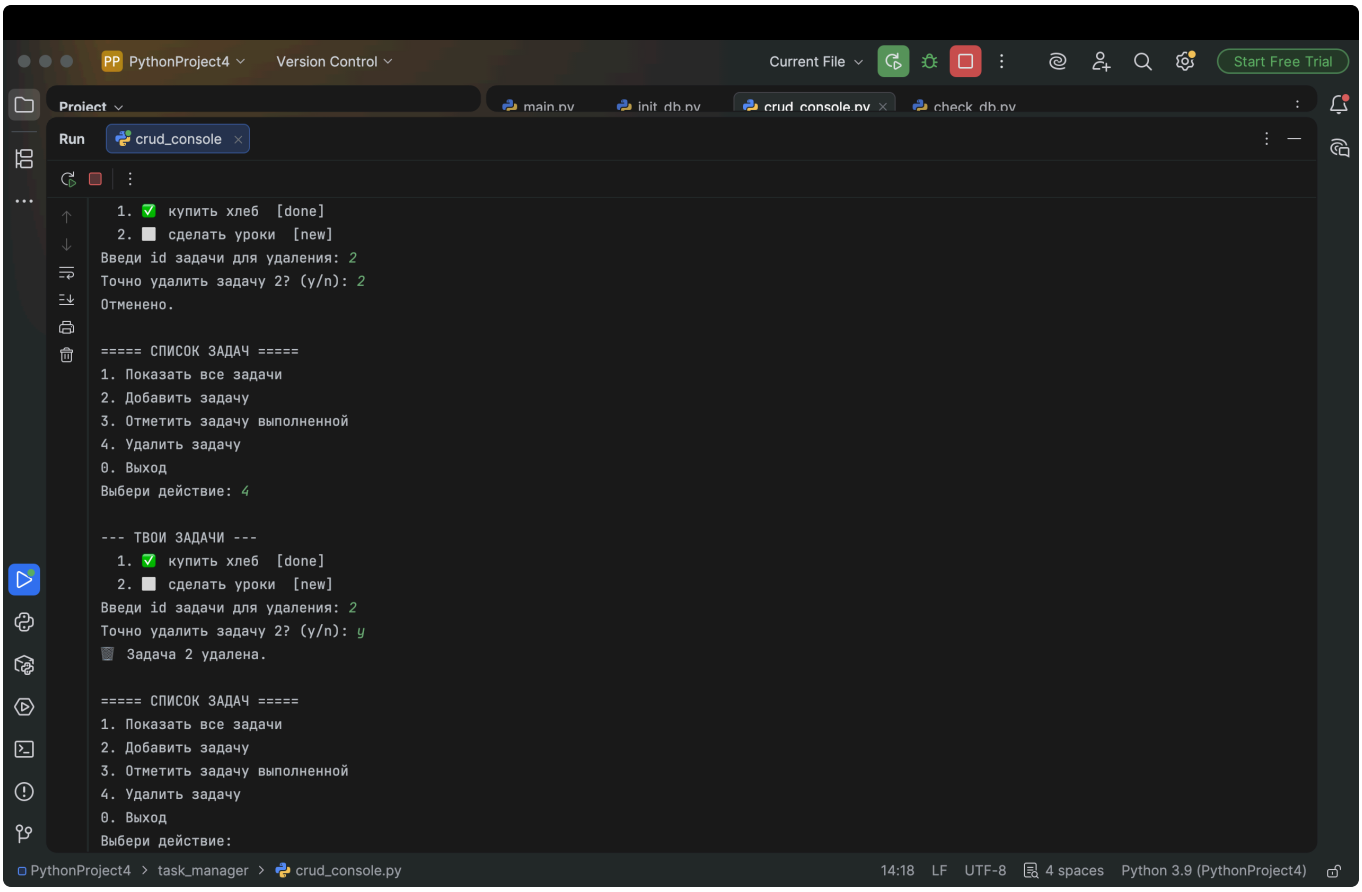
Шаг 8. Сценарий проверки

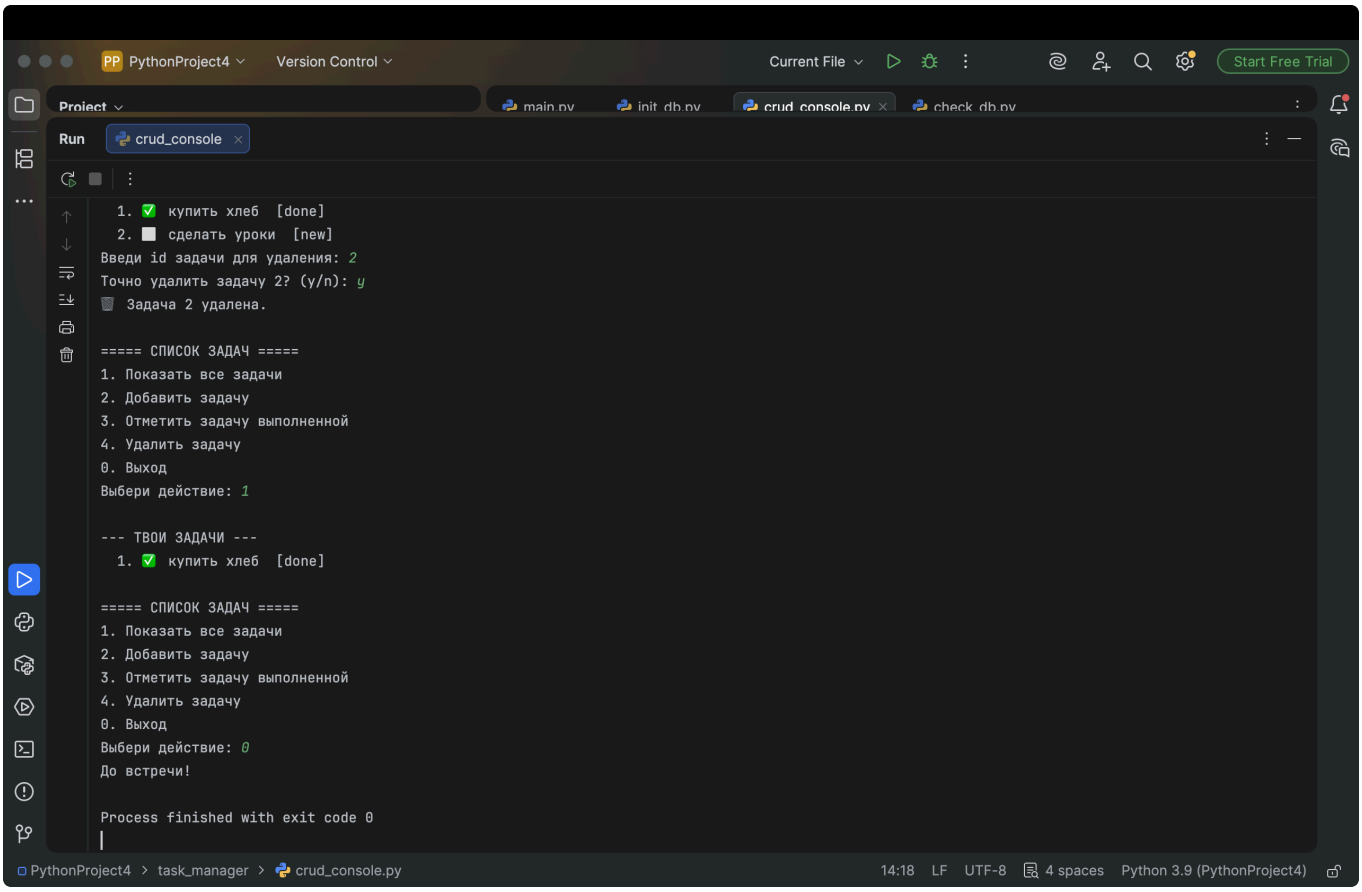



PythonProject4
Version Control
Current File
Start Free Trial
Project
PythonProject4 ~/PycharmProjects/PythonProject4
.venv
task_manager
main.py
init_db.py
crud_console.py
check_db.py
print(f"⚠ Задача с id={task_id} не найдена.")
else:
print(f"🗑 Задача {task_id} удалена.")
Run
crud_console
/Users/liliaastrebkova/PycharmProjects/PythonProject4/.venv/bin/python /Users/liliaastrebkova/PycharmProjects/PythonProject4/task_manager/crud_cons
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие: 1
Список пуст. Добавь первую задачу!
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие:
PythonProject4 › task_manager › crud_console.py
14:18 LF UTF-8 4 spaces Python 3.9 (PythonProject4)


PythonProject4
Version Control
Current File
Start Free Trial
Project
main.py
init_db.py
crud_console.py
check_db.py
Run
crud_console
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие: 2
Название задачи: купить хлеб
✅ Задача добавлена. Её id = 1
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие: 2
Название задачи: сделать уроки
✅ Задача добавлена. Её id = 2
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие:
PythonProject4 › task_manager › crud_console.py
14:18 LF UTF-8 4 spaces Python 3.9 (PythonProject4)


PythonProject4
Version Control
Current File
Start Free Trial
Project
main.py
init_db.py
crud_console.py
check_db.py
Run
crud_console
0. Выход
Выбери действие: 1
--- ТВОИ ЗАДАЧИ ---
1. ⬜ купить хлеб [new]
2. ⬜ сделать уроки [new]
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие: 3
--- ТВОИ ЗАДАЧИ ---
1. ⬜ купить хлеб [new]
2. ⬜ сделать уроки [new]
Введи id задачи, которую выполнил: 1
✅ Задача 1 отмечена выполненной.
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие:
PythonProject4 › task_manager › crud_console.py
14:18 LF UTF-8 4 spaces Python 3.9 (PythonProject4)


PythonProject4
Version Control
Current File
Start Free Trial
Project
main.py
init_db.py
crud_console.py
check_db.py
Run
crud_console
1. ✅ купить хлеб [done]
2. ⬜ сделать уроки [new]
Введи id задачи для удаления: 2
Точно удалить задачу 2? (y/n): 2
Отменено.
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие: 4
--- ТВОИ ЗАДАЧИ ---
1. ✅ купить хлеб [done]
2. ⬜ сделать уроки [new]
Введи id задачи для удаления: 2
Точно удалить задачу 2? (y/n): y
🗑 Задача 2 удалена.
===== СПИСОК ЗАДАЧ =====
1. Показать все задачи
2. Добавить задачу
3. Отметить задачу выполненной
4. Удалить задачу
0. Выход
Выбери действие:
PythonProject4 › task_manager › crud_console.py
14:18
LF
UTF-8
4 spaces
Python 3.9 (PythonProject4)



1. Почему для `SELECT` не нужен `commit()`, а для `INSERT` - нужен?

`SELECT` только читает данные и ничего не меняет в базе — сохранять нечего, поэтому фиксация не требуется. А `INSERT`, `UPDATE` и `DELETE` изменяют данные: до вызова `commit()` изменения находятся в буфере транзакции и не записаны в файл базы. Без `commit()` они откатятся при закрытии соединения и пропадут

2. Что вернёт `fetchone()`, если в таблице нет строк?

`None`

3. Чем `fetchone()` отличается от `fetchall()` и когда какой использовать?

`fetchone()` возвращает одну строку - кортеж значений, или `None`, если строк больше нет. `fetchall()` возвращает список всех строк результата запроса (пустой список, если строк нет). `fetchone()` используют, когда по логике задачи нужна ровно одна запись -например, `SELECT ... WHERE id = ?` для поиска конкретной задачи или проверка существования. `fetchall()` - когда нужны все записи сразу: «показать список задач», выгрузка для отчёта и т. п

4. Почему `(title,)` - это кортеж, а `(title)` - нет?

круглые скобки сами по себе — это оператор группировки, а не кортеж: `(title)` - это просто значение переменной `title`. Чтобы создать кортеж из одного элемента, нужна запятая: `(title,)`. Без неё `sqlite3` получит строку вместо кортежа параметров и выдаст ошибку `ProgrammingError: Incorrect number of bindings` - параметры должны передаваться именно в виде последовательности.

5. Что вернёт `cursor.rowcount` после `UPDATE`, если такого id нет?

`0`: запрос выполнится без ошибки, но ни одна строка не будет затронута.
Поэтому `rowcount` проверяют после выполнения: `0` - записи с таким id не существует, больше нуля - обновление прошло.

6. Зачем проверять `isdigit()` перед `int(input())`?

Пользователь может ввести не число - например, `abc` или пустую строку.
Функция `int()` в этом случае выбросит исключение `ValueError: invalid literal for int() with base 10` и программа упадёт. Проверка `isdigit()` заранее убеждается, что строка состоит только из цифр, и позволяет вместо падения программы показать аккуратное сообщение «id должен быть числом».

7. Как параметризованные запросы защищают от SQL-инъекции?

При параметризованном запросе значение передаётся не в текст SQL, а отдельно - как второй аргумент `execute()` через плейсхолдер `?`. SQLite подставляет его уже после разбора запроса и экранирует все специальные символы: кавычки, скобки, точку с запятой. Поэтому ввод вроде `'); DROP TABLE tasks; --` попадёт в базу как обычный текст названия задачи, а не выполнится как SQL-команда. При конкатенации строк (f-строки, `+`) этот же ввод стал бы частью SQL-кода - и таблица была бы удалена.